

Informe de reflexión CI/CD

Parte II · Pruebas, métricas propias y análisis de la práctica

Integrantes José Vanegas · Miguel Vanegas	Repositorio público github.com/migu145/inventario-app	Fecha 26-jul-2026
--	--	----------------------

01 Verificación reproducible

El README reúne los comandos en el orden en que se ejecutaron. Las salidas reales están en evidencias/capturas-reales, los registros de despliegue y el CSV de métricas. Con ese material, otra persona puede repetir las pruebas sin depender de explicaciones adicionales.

Elemento comprobado	Evidencia del repositorio	Resultado observado
Pipeline base	Secciones README 8 a 17; capturas 52 a 60	6 pruebas, build multi-stage, Actions en verde, GHCR y Rolling Update 2/2
Blue-Green	Capturas 65 y 66; blue-green-final.txt	Service en slot=green; respuesta v2/green; rollback a Blue verificado
Buenas prácticas	Capturas 59, 66, 67 y 68	Secret sin exponer la clave, Trivy sin CRITICAL y readiness 503 hasta quedar 1/1

02 Justificación de Blue-Green

Elegimos Blue-Green porque la aplicación expone /version con versión, color y hostname. Esto permite comprobar Green por separado y cambiar el tráfico modificando solo el selector slot del Service. Blue permanece activo, de modo que el rollback consiste en devolver el selector a slot=blue.

Canary habría servido para repartir solicitudes entre versiones, pero habría complicado la demostración con una muestra pequeña. Blue-Green consume más recursos porque mantiene dos versiones completas, pero el corte y el regreso a Blue son claros y reproducibles.

03 Persistencia al recrear el pod

El producto creado desde la interfaz quedó en /app/data/products.json dentro del contenedor que atendió la solicitud. Al eliminar ese pod, Kubernetes creó otro desde la imagen y el producto desapareció. Además, cada réplica conserva su propia copia, por lo que dos solicitudes pueden mostrar catálogos distintos.

Conclusión: el Deployment repone procesos, pero no comparte ni conserva los archivos de los contenedores. Un entorno real necesitaría almacenamiento persistente o una base de datos común.

04 Métricas DORA con datos propios

Git aporta la hora del commit, GitHub registra la finalización de cada run y Kubernetes confirma cuándo la imagen quedó disponible en el clúster. El lead time termina en el despliegue real, no en la publicación de GHCR. Todas las horas están en UTC-05.

00:02:55 · ÉLITE	2 por día · ÉLITE	33,33 % · MEDIO
Lead time promedio	Frecuencia de despliegue	Change failure rate

#	SHA	Commit	Run correcto	En el clúster	Lead time
1	8a26dc3	25-jul 16:57:29	Run 30176640875 16:59:15	25-jul 17:01:00	00:03:31
2	85eaa0f	25-jul 17:02:01	Run 30176786798 17:03:10	25-jul 17:04:20	00:02:19

Cálculos. Dos promociones correctas en un día equivalen a 2 despliegues/día. El promedio de 00:03:31 y 00:02:19 es 00:02:55. Hubo tres intentos: uno necesitó corrección y dos terminaron bien. La frecuencia cuenta los dos cambios que sí quedaron corriendo; el CFR incluye los tres intentos. Por eso, $1 / 3 \times 100 = 33,33 \%$.

Niveles. El lead time menor de una hora y los dos despliegues diarios sitúan la velocidad en ÉLITE. El CFR de 33,33 %, dentro del rango didáctico de 31 % a 45 %, deja la estabilidad en MEDIO. La muestra contiene solo tres intentos, por lo que esta clasificación es orientativa.

Fuentes: Git, runs 30176640875 y 30176786798, evidencias/despliegue-8a26dc3.txt, evidencias/despliegue-85eaa0f.txt y evidencias/dora-deployments.csv.

05 Problemas reales y resolución

Problema observado	Diagnóstico y solución aplicada
BuildKit omitía la etapa de pruebas.	Producción no dependía de test. Ahora copia el código desde esa etapa y el build exige las seis pruebas.
Una etiqueta inexistente causó ImagePullBackOff.	Se verificó GHCR, se publicó un SHA válido y se repitió el rollout hasta obtener 2/2 réplicas.
/health respondía 200 durante el arranque.	Se implementó STARTUP_DELAY_SECONDS. Ahora devuelve 503 durante la espera y 200 cuando el pod está listo.
Trivy agotó el tiempo al descargar su base.	Se usó el repositorio público alternativo. El pipeline final terminó sin hallazgos CRITICAL.
PowerShell alteraba el JSON de kubectl patch.	Los parches se guardaron en archivos y se aplicaron con --patch-file.

06 Reflexión final

La práctica nos obligó a comprobar cada afirmación con una salida real. Vimos que una imagen publicada no cuenta como desplegada hasta que Kubernetes la ejecuta, que un pod nuevo no recupera archivos locales y que un rollout correcto depende tanto de la imagen como del readiness. El proceso terminó rápido, pero el CFR muestra una mejora concreta: validar la existencia del SHA en GHCR antes de tocar el Deployment.